

LLM-Powered Research Agent – SOC



👋 **Welcome!**

Hey everyone!

This page will be your central resource hub throughout the Summer of Code program. I Prince (Elec) with Anushka(Civil) and Swaprabha(EP) , will be your mentors for this journey. Together, we'll build an AI agent that reads research papers, extracts knowledge, and generates literature reviews or hypotheses using LLMs.

We'll update this page weekly with new goals, resources, and tasks. Let's learn, build, and ship something awesome

Week 1-2: PDF Parsing + Embedding Basics

Goal of the Week:

- **Parse research papers (PDFs)**

- Before doing anything smart, we need to **extract the raw text** from the PDFs of research papers.

We'll use tools like:

- `PyMuPDF` or
- `pdfplumber`

Goal: Convert a research paper into raw text (title, abstract, paragraphs, sections).

- **Clean + chunk text**

- Once we have the text:
- Clean it (remove headers, footers, strange symbols, blank spaces)
- **Chunk** it: break the long document into smaller parts (like paragraphs or 200-word sections)

This is important because LLMs and vector databases work better with smaller, digestible pieces.

→Example chunk:

"In this study, we explore how transformer models can be used for document understanding..."

Use tools like:

- LangChain's `RecursiveCharacterTextSplitter`
 - LlamaIndex's document chunkers
-

- **Generate embeddings**

- Now, we convert each chunk of text into a **vector** (a long list of numbers) that captures its meaning — called an *embedding*.

You can use:

- OpenAI Embeddings (`text-embedding-3-small`)

- Sentence Transformers (`all-MiniLM-L6-v2` , BGE models)

Why embeddings? So that similar text has similar vectors → which lets us later **search** by meaning, not just keywords.

- **Store them in a vector DB (FAISS or Weaviate)**

- We now store the chunks and their embeddings in a **vector database**.
- **FAISS**: simple, fast, runs locally
- **Weaviate**: more powerful, web-based, supports filters & metadata

These databases let us **search** for relevant text chunks later, based on a user's question.

Later on:

When the user asks: "What methods did this paper use?" →

We convert the question to an embedding → find similar chunks in the DB
→ send those chunks to GPT to answer.

Tools & Libraries(week-1&2):

→Parse research papers (PDFs)

- PyMuPDF – for PDF text extraction
 - <https://pymupdf.readthedocs.io/en/latest/tutorial.html>
 - <https://youtube.com/playlist?list=PLHlrXTRZkTLBQ7k06CFoP3-gayAmfp-GG&si=B6WoVnDMqkx0w130>
- pdfplumber – alternative parser
 - <https://azhar-sayyad.medium.com/a-step-by-step-guide-to-parsing-pdfs-using-the-pdfplumber-library-in-python-c12d94ae9f07>
 - <https://www.youtube.com/watch?v=nk7e-ld6O3E>

→LangChain Text Splitters – for chunking

- https://python.langchain.com/docs/concepts/text_splitters/
- <https://medium.com/@harsh.vardhan7695/mastering-text-splitting-in-langchain-735313216e01>
- <https://www.youtube.com/watch?v=8OJC21T2SL4>
- <https://www.youtube.com/watch?v=SEWS9P4ODmc>
- <https://www.youtube.com/watch?v=lo43kf0hYn4>

→ Embeddings (Vectorization)

- <https://platform.openai.com/docs/guides/embeddings>
- <https://sentence-transformers.com/>
- <https://www.youtube.com/watch?v=OlhNZg4gOvA>
- <https://www.youtube.com/watch?v=bbEndo6dZyA&list=PLdF3rLdF4ICScQkCs5SKFO9zjihSpS8EN>
- <https://www.youtube.com/watch?v=eVLiKPcCN2k>
- Try: `all-MiniLM-L6-v2` or `bge-small-en`

→ Vector DB (Storing + Searching Chunks)

- [FAISS Quickstart](#)
- <https://www.youtube.com/watch?v=ZCSslkyCZk4>
- <https://www.youtube.com/watch?v=hSuCT6Z2QLk>
- <https://python.langchain.com/docs/integrations/vectorstores/faiss/> - LangChain FAISS Guide

Mini Task:

Upload a research paper → Parse it → Chunk it → Embed → Store in FAISS

Pro Tip:

Save each chunk with metadata (page number, section) to track where answers come from.

Week 3-4 – LangChain & Prompting Basics

→ Goal of the Week

- What **LangChain** is and how it works
- How to write good **prompts** for LLMs
- How to connect these two to build smart AI apps

Part 1: LangChain Basics

→ What is LangChain?

LangChain is a Python framework that helps you:

- Connect to LLMs like GPT
- Load and split documents
- Retrieve useful text chunks
- Build chains for question-answering

→ What to Learn

- How to use a `DocumentLoader` to load text
- How to split text using `TextSplitter`
- How to create chains using `LLMChain` or `RetrievalQA`

Resources

- <https://python.langchain.com/docs/tutorials/>
- <https://www.geeksforgeeks.org/introduction-to-langchain/>
- https://www.youtube.com/watch?v=pSVk-5WemQ0&list=PLKnIA16_RmvaTbihpo4MtzVm4XOQa0ER0
- <https://www.youtube.com/watch?v=swCPic00c30>
- https://www.youtube.com/watch?v=4O1rs7mrNDo&list=PLZoTAE LR MXVORE4VF7WQ_fAI0L1Gltar

Part 2: Prompting Basics

What is Prompting?

Prompting is how you “talk” to LLMs. The better your instruction, the better the output.

→ What to Learn

- Simple prompt formatting
- Using input variables (like `{context}` and `{question}`)
- Grounding prompts with context chunks
- <https://platform.openai.com/docs/guides/text?api-mode=responses>
- <https://www.youtube.com/watch?v=t2bSApmPzU4>

Mini Task

Try one of the following

- Load a `.txt` file in LangChain → Split it → Print the chunks
- Write a simple prompt like:

Use the following context to answer the question:

`{context}`

Question: `{question}`

- Ask GPT to answer using your prompt and a sample context

(you can use any other LLM also)

Assignment 1

- Implement a Retrieval-Augmented Generation (RAG) pipeline in Google Colab or Jupyter Notebook using LangChain to answer questions from the first three chapters of Zero to One by Peter Thiel.

book/novel that can be used for assignment –

<https://drive.google.com/file/d/1MmaUSvI55D40zMkmJv431sqnbFruD-Co/view?usp=sharing>

Deadline - friday

Week 5-6 – Streamlit & LangGraph implementation

→ Goal of the Week

- learn about Streamlit from documentation
- implement LangGraph

Part 1: Streamlit

→ Fundamental concepts and uses

- Built entirely in Python
- Auto-updates on input change

→ Resources

- fundamental concepts - <https://docs.streamlit.io/get-started/fundamentals>

- Streamlit API cheat sheet - <https://docs.streamlit.io/develop/quick-reference/cheat-sheet>
- About Streamlit - <https://docs.snowflake.com/en/developer-guide/streamlit/about-streamlit>
- https://youtu.be/yKTEC1Y5bEQ?si=6xFxG2m_sduS_qKC

LangGraph

LangGraph is a powerful framework built on top of LangChain that helps you create **stateful, multi-step applications** using **graph-like structures**.

Instead of writing long chains or workflows manually, LangGraph allows you to **define each step as a node** and connect them like a flowchart. This makes it easier to manage **complex logic**, handle **loops**, and add **memory or conditions** in your LLM-based applications.

You can think of it like building a **custom chatbot or agent flow**, where the logic is flexible and easy to control.

- **LangGraph**

Build controllable agents with LangGraph, our low-level agent orchestration framework. Deploy and scale with LangGraph Platform, with APIs for state management, a

 <https://www.langchain.com/langgraph>



- https://www.youtube.com/watch?v=jGg_1h0qzaM&t=4s
- https://www.youtube.com/watch?v=vVGXPRjtAJE&list=PLZoTAELRMXVPFd7JdvB-rnTb_5V26NYNO
- <https://www.youtube.com/watch?v=5h-JBkySK34&list=PLfaIDFEXuae16n2TWUkKq5PgJ0w6Pkwtg>

Final Product Description – *LLM-Powered Research Agent*

By the end of this project, you should have built a complete **LLM-based Research Chatbot** as the **core deliverable**.

✅ **Must-Have (Core Functionality):**

A **simple and functional research assistant** that:

- Lets users **upload a research paper (PDF)**
- Automatically **generates a summary**
- Allows users to **chat with the paper** using a conversational RAG pipeline
- Uses a **free LLM** (like Groq models) and a **vector store** (FAISS/Weaviate)

This will form your **minimum working product** and showcase how AI can assist in research understanding.

🌟 **Add-Ons (Optional but Encouraged):**

Explore integrating **LangGraph-based agents** as additional features:

- **Smart Summarizer Agent:** Summarizes different sections (abstract, methods, results) step-by-step
- **Literature Comparator Agent:** Compare multiple uploaded papers across key dimensions
- **Citation Recommender Agent:** Suggests related papers from arXiv/Semantic Scholar
- **Experiment Extractor Agent:** Extracts datasets, metrics, and key findings in structured form
- **Paper Quality Evaluator Agent:** Scores papers on clarity, novelty, soundness, etc.

You're free to design creative graphs using LangGraph, the goal is to turn your chatbot into a **modular research assistant**.

Thank you all for being with us in this project. It's been amazing to see everyone build, learn, and grow together.